

Web 2

Trabajo Practico N°0

Profesores: Matias Endres
Rodrigo Silva

Alumno: Nery José Lazarini Cardona

Respuestas

1. Del cliente al servidor y de HTTP a REST

1. Cliente: Genera la consulta pidiendole al servidor la informacion y muestra la respuesta en una interfaz.

Servidor: Recibe la consulta, la procesa, revisa las bases de datos (de haber) y retorna la respuesta al cliente en base a lo que solicito.

Ejemplo: Solicitud de una clase consulta. Yo pido a Walter una clase consulta, Walter va a recibir mi pedido, va a revisar su agenda y me va a devolver una fecha y hora para hacer la consulta.

2. API: Es el contrato o la interfaz. Son las reglas que definen como dos sistemas se pueden comunicar entre si.

HTTP: Es el medio de transporte, el protocolo de internet por donde viajan los mensajes.

REST: Es un estilo de arquitectura, una serie de buenas practicas para diseñar APIs sobre la web para que sean escalables y estandarizadas. Son distintas porque una API puede no usar HTTP o una API que si lo use pero no aplique la reglamentacion REST.

3. Para que una API se considere REST debe:

- i. Separar la interfaz del almacenamiento de datos.
- ii. Poder cambiar todo el backend sin que el frontend se entere.
- iii. El servidor no guarda memoria de la sesion del cliente. Cada request tiene que llevar toda la informacion necesaria para ser entendida. Esto facilita la escalabilidad porque cualquier servidor disponible puede atender cualquier request.
- iv. Las respuestas tienen que avisar si se pueden guardar en cache o no.
- v. Se usan estandares fijos para que la API sea predecible.
- vi. Ejemplo de API HTTP que no es REST: Un endpoint de estilo RPC (Remote Procedure Call). Supongamos que hay un endpoint que sea POST /modificarDatosDeUsuario. Usa HTTP para viajar, pero no es REST porque ignora el concepto de recursos, no usa el metodos de forma correcta, e incluye el verbo de accion en la URL

2. Principios SOLID en un backend

1.

- i. S - Single Responsibility Principle (Principio de Responsabilidad Unica): Una clase tiene que tener un solo motivo para cambiar, o sea, dedicarse a una sola cosa.

Ejemplo backend: Una clase UsuarioRepository solo se encarga de guardar y buscar usuarios en la base de datos. No deberia

tener código para encriptar contraseñas ni para mandar emails de bienvenida.

- ii. O - Open/Closed Principle (Principio de Abierto/Cerrado): El código tiene que estar abierto para sumar funcionalidades nuevas, pero cerrado para modificar el código que ya funciona. Ejemplo backend: Si a un procesador de pagos hay que sumarle MercadoPago, en vez de meter un if gigante en la clase vieja y romperla, creamos una nueva clase que implemente una interfaz común llamada MetodoDePago.
 - iii. L - Liskov Substitution Principle (Principio de Sustitución de Liskov): Si se usa herencias o interfaces, la clase hija tiene que poder reemplazar a la clase padre sin romper el programa. Ejemplo backend: Si a una interfaz BaseDeDatos con un método desconectar(), le hacemos una implementación para una base en memoria que no requiere desconectarse, ese método no debería romper la app ni tirar un error; debería no hacer nada, pero respetar el contrato original.
 - iv. I - Interface Segregation Principle (Principio de Segregación de Interfaces): Es mejor tener muchas interfaces chicas y específicas que una sola gigante que obligue a las clases a implementar métodos al vacío. Ejemplo backend: En vez de tener una interfaz IUserario con métodos comprar() y modificarProductos(), creamos un ICliente y IAdministrador. Así un usuario normal no tiene el método de modificar productos vacío o tirando error.
 - v. D - Dependency Inversion Principle (Principio de Inversión de Dependencias): Los módulos de alto nivel no deben depender de los de bajo nivel directamente, sino de abstracciones (interfaces). Ejemplo backend: Si a una clase UserService no hace un new MySqlConnection(). En su lugar, recibe por el constructor una interfaz DatabaseConnection. De esta forma podemos cambiar a PostgreSQL sin complicaciones.
2. Si una sola parte del código se encarga de validar pedidos, aplicar las reglas del negocio, buscar cosas en la base de datos y además mandar mails, se está rompiendo la regla de que cada cosa debe tener una sola responsabilidad. Para arreglar esto deberías separar esa pieza gigante en varias más pequeñas y específicas, donde una solo valide, otra solo busque datos y otra solo mande mensajes. Después conectar todo usando el principio de inversión de dependencias para que sea mucho más fácil hacer pruebas a cada parte por separado. Aplicar estas reglas de diseño no significa que haya que armar interfaces y estructuras complejas para absolutamente todo lo que se programe.

3. El lenguaje de HTTP

1. Es importante aclarar que ademas de estos metodos estandar existen los metodos de extension. Estos son verbos adicionales que se pueden crear para extender el protocolo HTTP segun las necesidades especificas de alguna aplicacion o de otros protocolos que funcionan por encima

Metodo	Finalidad	Ejemplo	Seguro	Idempotente
GET	Pide una representacion del recurso	/usuarios/1	Si	Si
HEAD	Pide lo mismo que GET pero sin el cuerpo de la respuesta, solo los encabezados	/usuarios/1	Si	Si
POST	Manda datos al servidor para crear un recurso nuevo o procesar algo	/usuarios	No	No
PUT	Reemplaza todo el recurso actual con los datos nuevos que mandas	/usuarios/1	No	Si
PATCH	Modifica solo una parte del recurso	/usuarios/1	No	No
DELETE	Borra el recurso	/usuarios/1	No	Si

	especifico			
CONNECT	Establece un tunel hacia el servidor identificado por el recurso objetivo	servidor.com	No	No
OPTIONS	Describe las opciones de comunicacion para el recurso	/usuarios	Si	Si
TRACE	Hace una prueba de bucle invertido a lo largo de la ruta al recurso	/usuarios	Si	Si

- Usar GET para modificar informacion en una base de datos es incorrecto porque este metodo esta diseñado para ser seguro y unicamente leer datos sin alterarlos. Si se rompe esta regla se generan problemas con los mecanismos de la web. Primero, las respuestas GET se guardan en cache, por lo que una solicitud de modificacion podria no llegar al servidor si se usa una version almacenada previamente. Segundo, ante fallas de red, los sistemas realizan reintentos automaticos con GET asumiendo que no alteran el estado, lo que provocaria que la modificacion se ejecute multiples veces. Por ultimo, los rastreadores automaticos y las funciones de prefetching de los navegadores acceden a las rutas GET por su cuenta para indexar o precargar contenido; si estas rutas modifican informacion, se alterarian o borrarían datos automaticamente con solo escanear la pagina.
- Los codigos de estado indican el resultado de una peticion y se dividen en cinco familias. La familia 1xx es informativa y señala que el servidor recibio la peticion. La 2xx indica exito en la operacion. La 3xx es de redireccion y señala que el recurso esta en otra ubicacion. La 4xx

representa errores del cliente al enviar solicitudes invalidas o no permitidas. La 5xx indica errores del servidor al intentar procesar la solicitud. Sobre los codigos mas comunes, el 200 indica que la peticion se resolvio con exito. El 201 señala que la peticion tuvo exito y se creo un recurso. El 204 indica exito pero sin contenido para devolver en la respuesta. El 400 señala una solicitud con sintaxis invalida. El 401 indica que se requiere autenticacion. El 403 señala que se denego el acceso por falta de permisos. El 404 indica que el recurso solicitado no existe. El 409 señala un conflicto con el estado actual del recurso. El 422 indica que la sintaxis es correcta pero hay errores semanticos en los datos. El 429 señala que se enviaron demasiadas peticiones en un tiempo determinado. El 500 es un error interno generico del servidor. El 503 indica que el servidor no esta disponible temporalmente para procesar la peticion.

4. Estado, Identidad y Permisos

1. Las aplicaciones stateful conservan el estado de la sesion en la memoria del propio servidor. Su principal ventaja es el acceso rapido a los datos de la sesion activa, pero su costo es que dificultan la escalabilidad horizontal y consumen mas memoria, ya que obligan a redirigir las peticiones del mismo usuario siempre al mismo servidor. Un ejemplo de stateful es un sistema donde un carrito de compras se guarda en la memoria del servidor web local. En cambio, las aplicaciones stateless conservan el estado del lado del cliente, quien debe enviarlo en cada peticion. Su ventaja es la alta escalabilidad porque cualquier instancia del servidor puede procesar la peticion, y su costo es el aumento del tamaño de cada solicitud en la red. Un ejemplo de stateless es una API REST donde el cliente envia un token de acceso en el encabezado de cada consulta. Un backend stateless si puede consultar una base de datos; el termino stateless significa que no se guarda el contexto de la conexion temporal del usuario en la memoria de la aplicacion, pero los datos permanentes del negocio siguen almacenandose en la base de datos.
2. La autenticacion es el proceso de verificar la identidad del usuario para confirmar que realmente es quien dice ser, mientras que la autorizacion se encarga de determinar si esa identidad verificada tiene los privilegios necesarios para realizar una accion especifica. Por ejemplo, en un sistema de gestion, la autenticacion se completa cuando ingresas tu correo y contraseña para acceder. Una vez dentro, la autorizacion evalua los permisos asociados a tu rol: si el rol es de empleado, el sistema te otorga permiso solo para visualizar reportes,

pero si el rol es de gerente, el sistema te da el permiso adicional para modificar y eliminar esos reportes.

3. Un JWT esta compuesto por tres partes: el header que indica el algoritmo de firma, el payload que contiene los datos, y la firma. La firma protege la integridad de los datos, garantizando matematicamente que ni el header ni el payload fueron modificados en transito por un tercero. Base64URL no cifra el contenido porque solo es un estandar de codificacion que convierte datos binarios en caracteres de texto seguros para viajar por internet; al no ser un metodo de encriptacion, cualquier persona con acceso al token puede decodificarlo y leer su payload de forma directa. Al recibir un JWT, el servidor debe validar la firma criptografica utilizando su clave secreta y verificar que la fecha de expiracion del token no haya pasado. La mayor dificultad para revocar estos tokens es que el servidor no lleva un registro de sesiones activas; para anular un token antes de su expiracion natural, es necesario crear y consultar una lista negra en una base de datos externa, lo cual rompe el modelo de arquitectura stateless y suma demoras en cada peticion.

5. Control de trafico y reintentos seguros.

1. El rate limiting resuelve el problema de la sobrecarga de los servidores y previene abusos o ataques limitando la cantidad de solicitudes que un cliente puede enviar en un periodo especifico. Una estrategia para aplicarlo es la ventana fija, donde el tiempo se divide en bloques exactos y se cuenta cuantas peticiones se reciben, reseteando el contador al terminar el bloque. Otra estrategia es el token bucket, donde se asigna una capacidad maxima de tokens que se consumen con cada solicitud y se regeneran a un ritmo constante a lo largo del tiempo.
2. Consultar el archivo Respuesta5_2.java para el codigo
Al implementar este control en la memoria local ocurren dos problemas. Si se ejecutan multiples instancias de la aplicacion utilizando un balanceador de carga, los contadores quedan aislados en cada servidor, permitiendo que las peticiones burlen el limite global al distribuirse en distintas instancias. Por otro lado, limitar estrictamente por direccion IP genera problemas en redes compartidas donde muchos usuarios legitimos navegan bajo la misma conexion, provocando que se bloqueen entre si al agotar el cupo rapidamente.
3. La idempotencia es la propiedad que asegura que ejecutar una operacion varias veces produce exactamente el mismo resultado final en el sistema que ejecutarla una sola vez. GET es idempotente al limitarse a leer informacion sin alterarla. PUT es idempotente porque reemplaza el recurso completamente dejando siempre el mismo estado final. DELETE es idempotente dado que al borrar un recurso el

resultado constante es su ausencia. POST no es idempotente porque peticiones repetidas crearan multiples recursos nuevos. PATCH tampoco es idempotente por defecto, ya que las modificaciones parciales aplicadas varias veces pueden generar estados diferentes. Al procesar transacciones se utiliza un encabezado llamado Idempotency-Key para prevenir operaciones duplicadas ante fallas de red. El cliente adjunta una clave unica a su solicitud; si la conexion se interrumpe y se reintenta la peticion, el servidor reconoce la clave, detecta que la transaccion ya fue procesada y devuelve el resultado exitoso original sin volver a ejecutar la operacion.

6. Contratos que crecen sin romper Clientes

1. Para el codigo consultar Respuesta6-1.json

La paginacion por pagina divide los resultados en bloques fijos calculados a partir de un numero de pagina y una cantidad de elementos, siendo muy intuitiva pero ineficiente en bases de datos inmensas. La paginacion por offset saltea una cantidad especifica de registros antes de empezar a devolver datos, lo que permite saltos exactos pero se vuelve lento en consultas profundas porque el sistema igual tiene que leer y descartar los primeros registros. La paginacion por cursor usa un puntero o identificador unico del ultimo registro visto para buscar directamente los siguientes, logrando un rendimiento excelente sin importar la profundidad, aunque tiene la desventaja de impedir saltar a paginas especificas al azar.

2. Las formas mas habituales de versionar una API son agregar el numero de version directamente en la URL, enviarlo a traves de un encabezado personalizado en la peticion HTTP, o especificarlo usando el encabezado de aceptacion mediante negociacion de contenido. Un cambio obliga a crear una nueva version unicamente cuando introduce modificaciones incompatibles hacia atras que romperian las aplicaciones de los clientes actuales, como por ejemplo eliminar un campo existente, cambiar el tipo de dato de una propiedad o exigir un parametro nuevo que antes no era obligatorio.

3. Un origen es la combinacion exacta del protocolo, el dominio y el puerto desde donde se hace una solicitud web. CORS es un mecanismo de seguridad de los navegadores que usa encabezados HTTP para permitir que un servidor indique a que otros origenes se les permite acceder a sus recursos. Un preflight es una peticion de prueba que hace el navegador por su cuenta usando el metodo OPTIONS para preguntarle al servidor si la operacion principal que quiere ejecutar esta permitida. Para que el frontend en <https://app.ejemplo.com> se pueda comunicar con la API en <https://api.ejemplo.com>, el servidor de la API tiene que incluir en sus respuestas un encabezado llamado Access-Control-Allow-Origin que

contenga exactamente la direccion del frontend. CORS no reemplaza a la autenticacion porque su unica funcion es restringir desde que paginas web un navegador puede cargar recursos cruzados, pero no tiene forma de verificar quien es la persona detras de la pantalla ni que nivel de acceso deberia tener sobre la informacion.

7. Elegir un modelo de datos

1. Las bases de datos SQL tienen una estructura rigida basada en tablas con esquemas fijos, mientras que las NoSQL son flexibles y guardan la informacion como documentos o pares clave-valor sin un esquema definido. Para las relaciones, SQL es ideal porque permite conectar tablas de forma eficiente usando claves foraneas, algo que NoSQL prefiere evitar guardando toda la informacion relacionada junta o anidada. Al momento de consultar datos, SQL usa un lenguaje estandarizado muy potente para busquedas complejas, mientras que cada base NoSQL usa su propio formato mas enfocado en accesos directos y rapidos. En cuanto a las transacciones y la consistencia, SQL garantiza operaciones completamente seguras y consistencia inmediata para que los datos siempre sean exactos en todo el sistema, mientras que NoSQL suele priorizar la disponibilidad y la velocidad aceptando que los datos tarden un instante en sincronizarse. Finalmente para escalar, SQL crece de forma vertical necesitando un servidor cada vez mas potente, y NoSQL escala de forma horizontal repartiendo la carga facilmente al sumar mas servidores comunes.
2. Para un sistema universitario elegiria una base de datos SQL porque requiere relaciones complejas entre alumnos, materias y profesores, ademas de necesitar transacciones seguras para garantizar la consistencia absoluta en temas criticos como las notas o las inscripciones. Para un registro masivo de eventos usaria NoSQL porque esta diseñado para soportar un volumen inmenso de escrituras por segundo y permite escalar horizontalmente a medida que el trafico aumenta. Para un catalogo con atributos variables tambien elegiria NoSQL porque su flexibilidad permite guardar un producto con veinte características y otro con solo tres sin tener que modificar la estructura de la base de datos ni dejar espacios vacios. Una aplicacion perfectamente puede combinar ambos enfoques utilizando una arquitectura conocida como persistencia poliglota, donde por ejemplo usas SQL para manejar los usuarios y los pagos donde necesitas consistencia estricta, y al mismo tiempo usas NoSQL para el catalogo de productos o el registro de actividades buscando velocidad y flexibilidad.

8. Organizar y encontrar datos

1. Un índice en una base de datos es como el índice alfabético al final de un libro. En lugar de leer toda la base de datos fila por fila para encontrar un dato específico, el motor de la base de datos consulta el índice, que ya tiene la información ordenada y le dice exactamente en qué ubicación física está lo que busca, acelerando muchísimo las consultas. Sin embargo, no todo es gratis; los índices ocupan espacio extra en el disco y hacen que las operaciones de escritura, como insertar, actualizar o borrar datos, sean más lentas porque el sistema tiene que actualizar el índice cada vez que se cambia la información principal. Los tipos de índices más habituales son el B-Tree, que es el estándar y sirve para la mayoría de las búsquedas exactas o por rangos, y el Hash, que es muy rápido pero solo sirve para coincidencias exactas. Conviene usar un índice, por ejemplo, en la columna del correo electrónico de una tabla de usuarios, ya que el sistema la va a consultar constantemente cada vez que alguien intente iniciar sesión.
- 2.

9. Evitar trabajo repetido y acercar contenido

1. Consultar consulta en Respuesta9_1
El patrón cache-aside consiste en que la aplicación primero busca el dato mediante su clave en la cache (hit) y si no lo encuentra (miss) va a la base de datos para luego guardarlo en memoria con un tiempo de vida útil o TTL. Las principales limitaciones de usar cache en memoria local son que los datos se pierden si el servidor se reinicia, consume mucha memoria RAM si se guardan objetos grandes y en sistemas con múltiples instancias cada servidor tiene su propia cache aislada, lo que puede generar inconsistencias entre ellos.
2. Una CDN es una red de servidores distribuidos geográficamente para entregar contenido de internet más rápido. El servidor de origen es la computadora principal donde está alojada la versión original de la aplicación. Los servidores de borde son los equipos de la CDN ubicados en distintas ciudades cerca de los usuarios finales. El contenido habitual que se guarda en estas redes son archivos estáticos que no cambian seguido como imágenes, videos, fuentes o scripts. El beneficio principal es la reducción de la latencia ya que los usuarios descargan el contenido desde un servidor cercano a su ubicación en lugar de cruzar el mundo para conectarse al servidor de origen, además de reducir el tráfico en tu infraestructura principal.

Bibliografía:

1. <https://developer.mozilla.org/es/docs/Web/HTTP/Guides/Overview>
2. <https://www.redhat.com/es/topics/api/what-is-a-rest-api>
3. <https://www.freecodecamp.org/espanol/news/los-principios-solid-explicados-en-espanol/>
4. <https://developer.mozilla.org/es/docs/Web/HTTP/Methods>
5. <https://developer.mozilla.org/es/docs/Web/HTTP/Status>
6. <https://www.redhat.com/es/topics/cloud-native-apps/stateful-vs-stateless>
7. <https://auth0.com/es/intro-to-iam/authentication-vs-authorization>
8. <https://www.cloudflare.com/es-es/learning/bots/what-is-rate-limiting/>
9. <https://developer.mozilla.org/es/docs/Glossary/Idempotent>
10. https://docs.stripe.com/api/idempotent_requests
11. <https://learn.microsoft.com/es-es/azure/architecture/best-practices/api-design>
12. <https://developer.mozilla.org/es/docs/Web/HTTP/CORS>
13. <https://aws.amazon.com/es/nosql/>
14. <https://www.mongodb.com/es/nosql-explained>
15. <https://www.postgresql.org/docs/current/indexes.html>
16. <https://learn.microsoft.com/es-es/sql/relational-databases/sql-server-index-design-guide>
17. <https://aws.amazon.com/es/caching/database-caching/>
18. <https://www.cloudflare.com/es-es/learning/cdn/what-is-a-cdn/>